

基于 DLP 平台实现图像分类实验

卷积神经网络是实现图像分类、目标检测等计算机视觉任务的基础。本实验将介绍如何使用 pycnnl 库提供的 Python 接口，将 VGG19 网络模型移植到 DLP 平台上，实现图像分类，并分析比较 DLP 和 CPU 平台上运行 VGG19 进行图像分类的性能。

完成本实验，读者可以进一步理解 VGG19 网络在异构智能计算平台上的实现流程，掌握利用 DLP 平台进行卷积神经网络推断的方法。

3.2 基于 DLP 平台实现图像分类

3.2.1 实验目的

巩固卷积神经网络的设计原理，能够使用 pycnnl 库提供的 Python 接口将 VGG19¹ 网络模型移植到 DLP 上，实现图像分类。具体包括：

1. 使用 pycnnl 库实现卷积、ReLU 等基本网络模块。
2. 使用提供的 pycnnl 库实现 VGG19 网络。
3. 分析并比较 DLP 和 CPU 平台上运行 VGG19 进行图像分类的性能。

实验工作量：约需 1 个小时。

3.2.2 实验环境

硬件环境：DLP。

软件环境：pycnnl 库，Python 编译环境及相关的扩展库，包括 Python 3.6.12、Pillow 7.2.0、Scipy 1.2.0、NumPy 1.19.5。

数据集：ImageNet。

3.2.3 实验内容

本实验调用 DLP 平台上的 pycnnl 库来搭建 VGG19 网络进行图像分类。模块划分方式与第3.1节实验类似，分别为数据加载模块、基本单元模块、网络结构模块和网络推断模块。

3.2.4 实验步骤

3.2.4.1 数据加载模块

数据加载模块实现数据读取和预处理，如代码示例3.8所示。由于 Python 语言限制，调用 pycnnl 库的 Python 接口目前需要将数据类型从 numpy.float32 转换为 numpy.float64。

代码示例 3.8 VGG19 的数据加载模块的 DLP 实现示例

```
# file: vgg19_demo.py
def load_image(self, image_dir):
    # 读取图像数据
    self.image = image_dir
    image_mean = np.array([123.68, 116.779, 103.939])
    print('Loading and preprocessing image from ' + image_dir)
    input_image = scipy.misc.imread(image_dir)
    input_image = scipy.misc.imresize(input_image, [224, 224, 3])
    input_image = np.array(input_image).astype(np.float32)
```

```

input_image -= image_mean
input_image = np.reshape(input_image, [1] + list(input_image.shape))

self.input_data = input_image.flatten().astype(np.float)
# 将图片加载到 DLP 上
self.net.setInputData(input_data)

```

3.2.4.2 基本单元模块

VGG19 中包含的卷积层、ReLU 层、最大池化层、全连接层和 Softmax 层可以直接调用 `pycnnl` 库来实现对应层的初始化、参数加载、前向传播等操作。`pycnnl` 库的使用方式可以参考第2.2.1节的示例。

3.2.4.3 网络结构模块

与第2.5.3节类似，网络结构模块也使用一个类来定义 VGG19 网络，可以直接使用 `pycnnl` 封装好的基本模块接口来定义。网络结构模块在 DLP 上的实现如代码示例3.9所示，其中定义了以下成员函数：

- **神经网络初始化：**创建 `pycnnl.CnnlNet()` 的实例 `net`，建立神经网络结构；首先加载数据和权重的参数，然后调用 `net` 中创建网络层的接口定义整个神经网络的拓扑结构，并设定每层的超参数。

代码示例 3.9 VGG19 的网络结构模块的 DLP 实现

```

# file: vgg19_demo.py
class VGG19(object):
    def __init__(self):
        # 初始化网络，创建 pycnnl.CnnlNet() 实例 net
        self.net = pycnnl.CnnlNet()

    def build_model(self, param_path='../data/vgg19_data/imagenet-vgg-verydeep-19.mat'):
        self.param_path = param_path

        # TODO: 使用 net 的 createXXXLayer 接口搭建 VGG19 网络
        self.net.setInputShape(1, 3, 224, 224)

        # conv1_1
        input_shape1 = pycnnl.IntVector(4)
        input_shape1[0] = 1
        input_shape1[1] = 3
        input_shape1[2] = 224
        input_shape1[3] = 224
        self.net.createConvLayer('conv1_1', input_shape1, 64, 3, 1, 1)

        # relu1_1
        self.net.createReLuLayer('relu1_1')

        # conv1_2
        input_shape2 = pycnnl.IntVector(4)
        input_shape2[0] = 1
        input_shape2[1] = 64
        input_shape2[2] = 224
        input_shape2[3] = 224
        self.net.createConvLayer('conv1_2', input_shape2, 64, 3, 1, 1)

        # relu1_2
        self.net.createReLuLayer('relu1_2')

        # pool1
        # -----
        # -----

```

```

# -----
# fc8
input_shape3 = pycnnl.IntVector(4)
input_shape3[0] = 1
input_shape3[1] = 1
input_shape3[2] = 1
input_shape3[3] = 4096
weight_shape3 = pycnnl.IntVector(4)
weight_shape3[0] = 1
weight_shape3[1] = 1
weight_shape3[2] = 4096
weight_shape3[3] = 1000
output_shape3 = pycnnl.IntVector(4)
output_shape3[0] = 1
output_shape3[1] = 1
output_shape3[2] = 1
output_shape3[3] = 1000
self.net.createMlpLayer('fc8', input_shape3, weight_shape3,
output_shape3)
# Softmax
input_shapes = pycnnl.IntVector(3)
input_shapes[0] = 1
input_shapes[1] = 1
input_shapes[2] = 1000
self.net.createSoftmaxLayer('Softmax', input_shapes, 1)

```

3.2.4.4 网络推断模块

VGG19 网络推断模块在 DLP 上的实现如代码示例3.10所示。该模块同样划分为参数加载、前向传播、推断函数主体等操作，这些操作使用 VGG19 神经网络类的成员函数来定义：

- **神经网络参数的加载：**VGG19 网络参数包括卷积层和全连接层的权重和偏置。首先读取 VGG19 预训练模型文件，然后循环遍历 net 中的所有层；如果当前层是卷积或全连接层，则将对应的权重、偏置加载到层中。将模型文件读入内存之后，也需要做两方面的处理：一方面，训练得到的模型中权重维度为 $H \times W \times C_{in} \times C_{out}$ ，而 DLP 处理网络层时权重的维度为 $C_{out} \times H \times W \times C_{in}$ ，因此需要对读取的权重做一次维度交换，使其与 DLP 中权重的维度一致；另一方面，需要手动将 Numpy 数据类型转换为 np.float64 类型。
- **神经网络的前向传播：**输入经过预处理的图像之后，net.forward 函数会自动遍历调用 net 中的每一层的前向传播函数，并返回最后一层的结果。
- **神经网络推断函数主体：**与第3.1.5.4节的 CPU 实现类似，给定一张经过预处理的图像数据，执行网络的前向传播函数即可得到 VGG19 预测的 1000 个类别的分类概率，然后选取概率最高的类别作为网络最终预测的分类类别。

代码示例 3.10 VGG19 的网络推断模块的 DLP 实现

```

# file: vgg19_demo.py
def load_model(self): # 加载神经网络参数
    print('Loading parameters from file ' + self.param_path)
    params = scipy.io.loadmat(self.param_path)
    self.image_mean = params['normalization'][0][0][0]
    self.image_mean = np.mean(self.image_mean, axis=(0, 1))
    count = 0
    for idx in range(self.net.size()):
        if 'conv' in self.net.getLayerName(idx):

```

```

        weight, bias = params['layers'][0][idx][0][0][0][0]
        # matconvnet: weights dim [height, width, in_channel, out_channel]
        # ours: weights dim [out_channel, height, width, in_channel]
        weight = np.transpose(weight, [3, 0, 1,
2])).flatten().astype(np.float)
        bias = bias.reshape(-1).astype(np.float)
        self.net.loadParams(idx, weight, bias)
        count += 1
        if 'fc' in self.net.getLayerName(idx):
            weight, bias = params['layers'][0][idx - 1][0][0][0][0]
            weight = weight.reshape([weight.shape[0] * weight.shape[1] *
weight.shape[2], weight.shape[3]])
            weight = weight.flatten().astype(np.float)
            bias = bias.reshape(-1).astype(np.float)
            self.net.loadParams(idx, weight, bias)
            count += 1

def forward(self): # 神经网络的前向传播
    return self.net.forward()

def get_top5(self, label):
    # 打印推断的时间
    start = time.time()
    self.forward()
    end = time.time()
    print('inference time: %f' % (end - start))
    result = self.net.getOutputData()
    # 打印 top1/5 结果
    top1 = False
    top5 = False
    print('----- Top 5 of ' + self.image + ' -----')
    prob = sorted(list(result), reverse=True)[:6]
    if result.index(prob[0]) == label:
        top1 = True
    for i in range(5):
        top = prob[i]
        idx = result.index(top)
        if idx == label:
            top5 = True
        print('%f - %s' % (top, self.labels[idx].strip()))
    return top1, top5

def evaluate(self, file_list): # 推断函数主体
    top1_num = 0
    top5_num = 0
    total_num = 0
    # 读取标签
    self.labels = []
    with open('synset_words.txt', 'r') as f:
        self.labels = f.readlines()
    # 记录推断所有图片的总时间
    start = time.time()
    with open(file_list, 'r') as f:

```

```

file_list = f.readlines()
total_num = len(file_list)
for line in file_list:
    image = line.split()[0].strip()
    label = int(line.split()[1].strip())
    self.load_image(image)
    top1, top5 = self.get_top5(label) # 获取推断结果
    if top1:
        top1_num += 1
    if top5:
        top5_num += 1
end = time.time()
print('Global accuracy:')
print('accuracy1: %f (%d/%d)' % (float(top1_num) / float(total_num),
top1_num, total_num))
print('accuracy5: %f (%d/%d)' % (float(top5_num) / float(total_num),
top5_num, total_num))
print('Total execution time: %f' % (end - start))

```

3.2.4.5 实验完整流程

完成以上所有模块后，就可以调用上述模块中的函数，在 DLP 上运行 VGG19 网络实现给定图像的分类，如代码示例3.11所示。与第3.1节的 CPU 实现类似，首先实例化 VGG19 网络的类，其次建立网络结构，设置每层的超参数，然后读取模型文件为每层加载参数，最后输入待分类的图像并调用推断模块获得网络的分类结果。

代码示例 3.11 VGG19 进行图像分类的完整流程的 DLP 实现

```

# file: vgg19_demo.py
if __name__ == '__main__':
    vgg = VGG19()
    vgg.build_model()
    vgg.load_model()
    vgg.evaluate('file_list')

```

3.2.4.6 实验运行

根据第3.2.4.1节 - 第3.2.4.5节的描述补全 vgg19_demo.py 代码，并通过 Python 运行.py 代码。具体可以参考以下步骤。

1) 环境申请

申请实验环境并登录云平台，云平台上 `/opt/code_chap_2_3/code_chap_2_3_student/` 目录下是本实验的示例代码。同时，还需要加载 pycnnl。

```

# 登录云平台
ssh root@xxx.xxx.xxx.xxx -p xxxxxx
# 进入 code_chap_2_3_student 目录
cd /opt/code_chap_2_3/code_chap_2_3_student
# 进入 pycnnl/cnnl-python 目录下
cd pycnnl/cnnl-python
# 运行 build_pycnnl.sh 脚本文件，生成扩展模块
bash build_pycnnl.sh

```

2) 代码实现

补全 stu_upload 中的 vgg19_demo.py 文件。

```
# 进入实验目录
cd exp_3_1_vgg
# 补全 vgg19_demo.py
vim stu_upload/vgg19_demo.py
```

3) 运行实验

```
# 运行完整实验
python main_exp_3_2.py
```

3.2.5 实验评估

本实验仍然选择猫咪的图像进行分类测试，该猫咪图像的真实类别为 tabby cat，对应 ImageNet 数据集类别编号的 281。若实验结果将该图像的类别编号判断为 281，则可以认为判断正确。性能评判标准为预测猫咪类别时 VGG19 网络 forward 函数运行的时间。

本实验的评分标准设定如下：

- **100 分标准：**使用 pycnnl 搭建 VGG19 网络，给定 VGG19 的网络参数值和输入图像，可以得到正确的 Softmax 层输出结果和正确的图像分类结果。

3.2.6 实验思考

在实验中请思考如下问题：

1. 阅读 pycnnl/src/net.cpp 中 forward 函数的实现，比较 DLP 在计算哪些层时比 CPU 要快，为什么？
2. 观察 forward 函数的实现，在 VGG19 网络的一次完整推断过程中，DLP 每执行完一层都需要和 CPU 交互一次，这种交互是否有必要？有什么办法可以避免这种交互吗？

1. VGG19 为经典卷积神经网络模型。 [🔗](#)